04

Part 4 · Operating at Scale & Interview Toolkit

The topics that separate good answers from great ones: performance, quality, orchestration, trade-offs, plus a design template, rapid-fire Q&A and glossary.

- Partitioning & small files
- Shuffle, skew & tuning
- Schema evolution & contracts
- Data quality & governance
- Orchestration (Airflow)
- Kafka · CAP · Sharding
- Data Vault & keys
- Design template · Q&A · Glossary

4.1 Partitioning, Clustering & the Small-File Problem

★★★

IN ONE LINE

Physically organising data so queries skip what they do not need.

Partitioning physically splits a table into folders by column values so queries read only the relevant slice (**partition pruning**). It is the cheapest big win in lake performance — and, done badly, the cheapest way to create a mess.

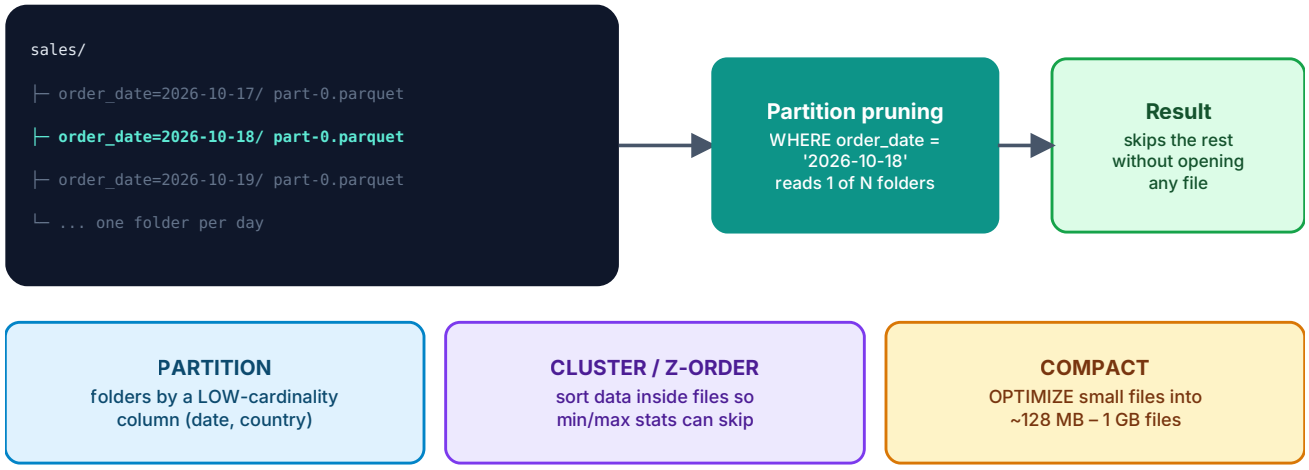


Figure 4.1 — Partition pruning, clustering and compaction

Concept	Rule of thumb
Partition column	Low cardinality, used in most filters (date is the classic). Aim for ≥ 1 GB of data per partition.
Over-partitioning	Partitioning by `user_id` or by hour on a small table creates millions of tiny files and slow metadata.
Small-file problem	Streaming and frequent micro-batches write many small files → slow listing, too many tasks. Fix with `OPTIMIZE` / auto-compaction.
Z-ORDER / liquid clustering	Co-locate rows with similar values in several columns (not just one) so min/max file statistics can skip files. Liquid clustering replaces static partitioning + Z-order and is changeable without rewriting.
Hidden partitioning	Iceberg derives partitions from column transforms (e.g. `days(ts)`) so queries do not need a separate partition column.

INTERVIEW ANGLE

Interview shortcut: **“partition by what you filter on, keep partitions big, cluster by high-cardinality filter columns, compact regularly.”**

4.2 Shuffle, Skew & Join Strategies

★★★

IN ONE LINE

Understanding data movement is the key to tuning distributed jobs.

Most slow Spark jobs come down to **shuffles**. A shuffle moves data across the network and disk so rows with the same key land together (joins, groupBy, distinct, orderBy). Reduce them, or make them cheaper.

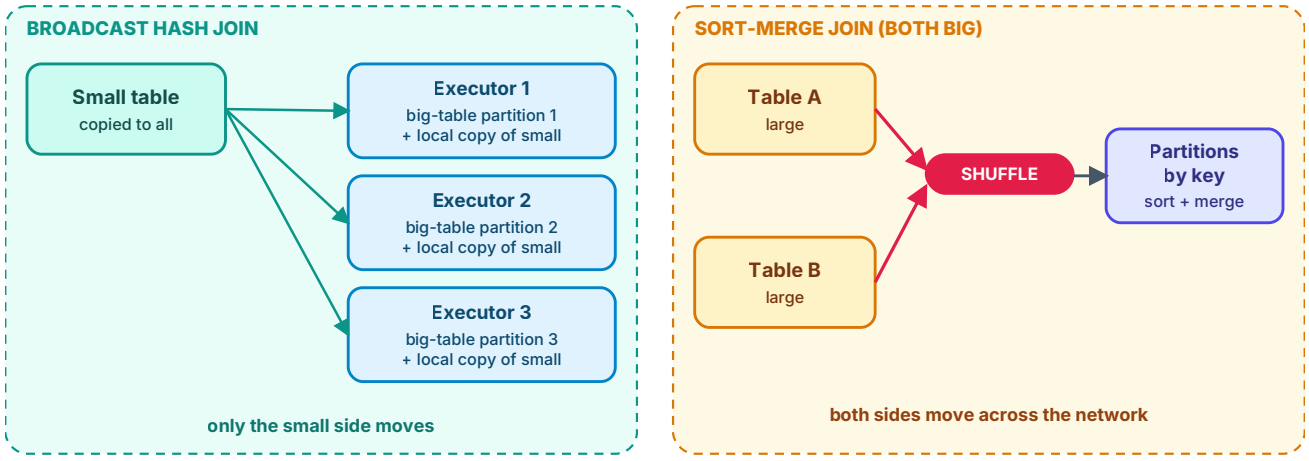


Figure 4.2a — Two join strategies: avoid shuffling the big side when you can

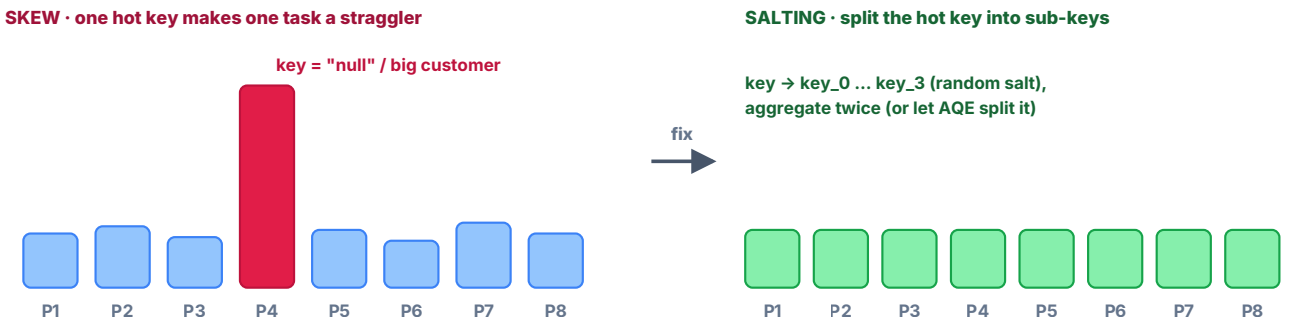


Figure 4.2b — Data skew and the salting fix

Problem	Symptom	Fixes
Shuffle-heavy join	Long stages, large shuffle read/write	Broadcast the small table (`broadcast(df)`, or auto-broadcast threshold); filter and select columns before the join
Data skew	Most tasks finish fast, one runs for ages	Enable **AQE skew-join handling**; salt the hot key; filter out null keys; isolate the hot key and broadcast it
Too few / too many partitions	OOM / spill, or thousands of tiny tasks	Tune `spark.sql.shuffle.partitions` (or rely on AQE coalescing); `repartition` before write
Spill to disk	Slow tasks, "spill" metrics	More memory per executor, fewer cores per executor, smaller partitions
Python UDFs	CPU-bound, serialisation overhead	Use built-in functions or pandas / Arrow UDFs

4.3 Schema Enforcement, Evolution & Data Contracts

★★☆

IN ONE LINE

Controlling how structure changes so downstream consumers do not break.

Source schemas change. A **schema-on-write** table **enforces** its schema (writes that do not match are rejected), while **schema evolution** lets you change it deliberately and safely. A **data contract** turns this into an agreement between producers and consumers.

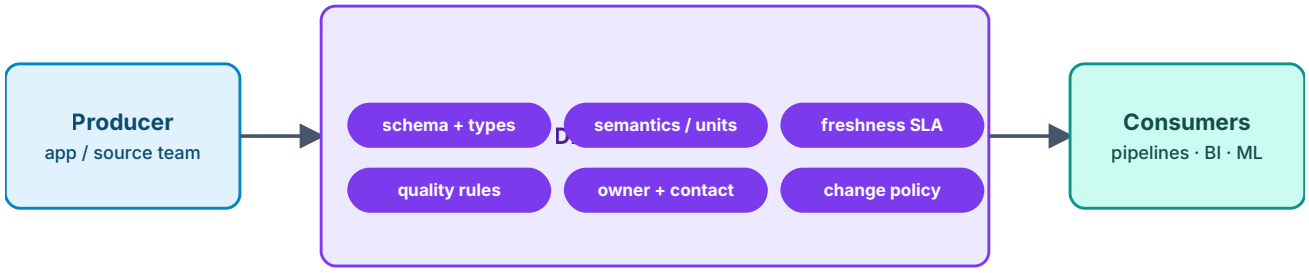


Figure 4.3 — A data contract sits between producer and consumers

Change	Backward compatible?	Typical handling
Add a nullable column	✓ yes	Schema evolution (e.g. Delta `mergeSchema`, Avro default values)
Widen a type (int → long)	✓ usually	Supported by many formats; verify downstream casts
Rename a column	× breaking	Column mapping (name-based), or add new + deprecate old
Drop a column	× breaking	Deprecate first, notify consumers, then remove
Change semantics / units	× silently breaking	New versioned field; contract review — the most dangerous kind

Schema enforcement vs evolution

Enforcement: reject bad writes (protects quality). **Evolution**: opt in to add columns (e.g. `.option("mergeSchema","true")`). Keep Bronze permissive (rescue unexpected columns) and Silver strict.

Tools

Schema registries for Kafka (Avro / Protobuf compatibility modes), dbt model contracts, Open Data Contract Standard, CI checks that fail PRs on breaking changes.

4.4 Data Quality, Observability, Lineage & Governance

★★★

IN ONE LINE

Making sure data is correct, fresh, traceable and appropriately accessed.

Pipelines that run successfully can still produce **wrong data**. Data quality checks, observability, lineage and governance are how teams find out before the business does.

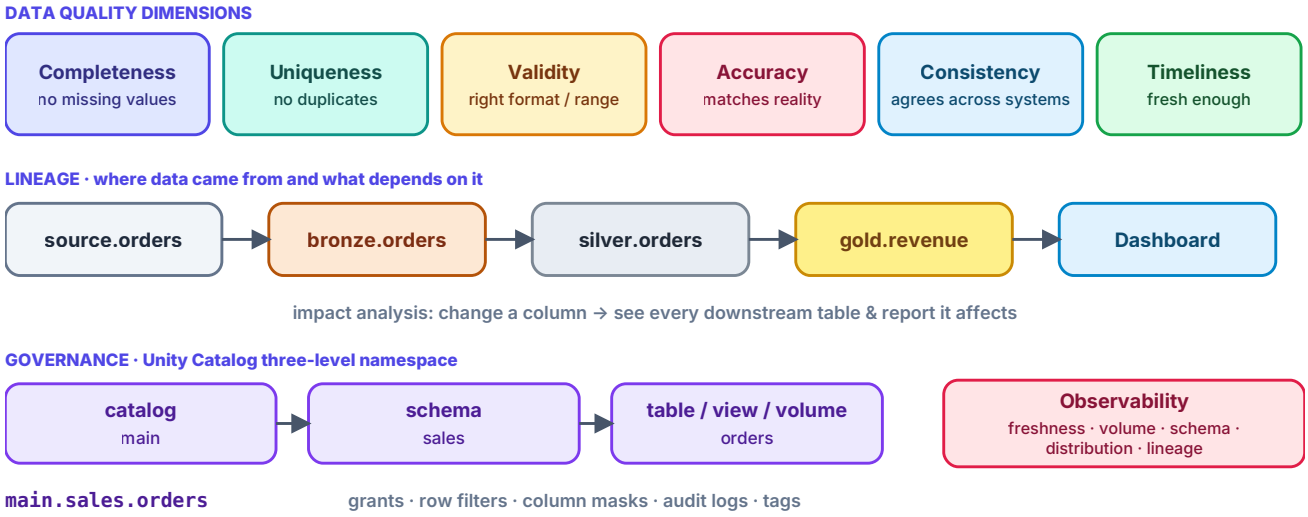


Figure 4.4 — Quality dimensions, lineage and catalog governance

Where to check

- **At ingestion**: schema, required fields, row counts vs source.
- **In transformation**: uniqueness, referential integrity, accepted values (dbt tests, DLT expectations).
- **After load**: reconciliation totals, freshness, anomaly detection.
- Route failures to a **quarantine** table, not the void.

Observability signals

- **Freshness** — did data arrive on time?
- **Volume** — unexpected row-count drop/spike.
- **Schema** — drift or breaking change.
- **Distribution** — null rate, value ranges shift.
- **Lineage** — who is impacted.

Tool	Role
dbt tests, DLT expectations, Great Expectations, Soda	Declarative data-quality rules in the pipeline
Unity Catalog, Glue Catalog, DataHub, Collibra, Alation	Catalog, governance, access control, lineage
Monte Carlo, Databand, Elementary	Data observability and anomaly detection

INTERVIEW ANGLE

A strong answer includes **what you do when a check fails**: fail fast for critical constraints, quarantine for row-level issues, alert the owner, and never silently publish bad data to Gold.

4.5 Orchestration (Airflow & DAGs)

★★★

IN ONE LINE

Scheduling, ordering, retrying and monitoring the tasks that make up pipelines.

An orchestrator schedules and coordinates tasks: it runs them in dependency order, retries failures, tracks state, supports backfills and alerts people. **Apache Airflow** is the best-known; alternatives include Dagster, Prefect, Databricks Workflows, Azure Data Factory and AWS Step Functions.

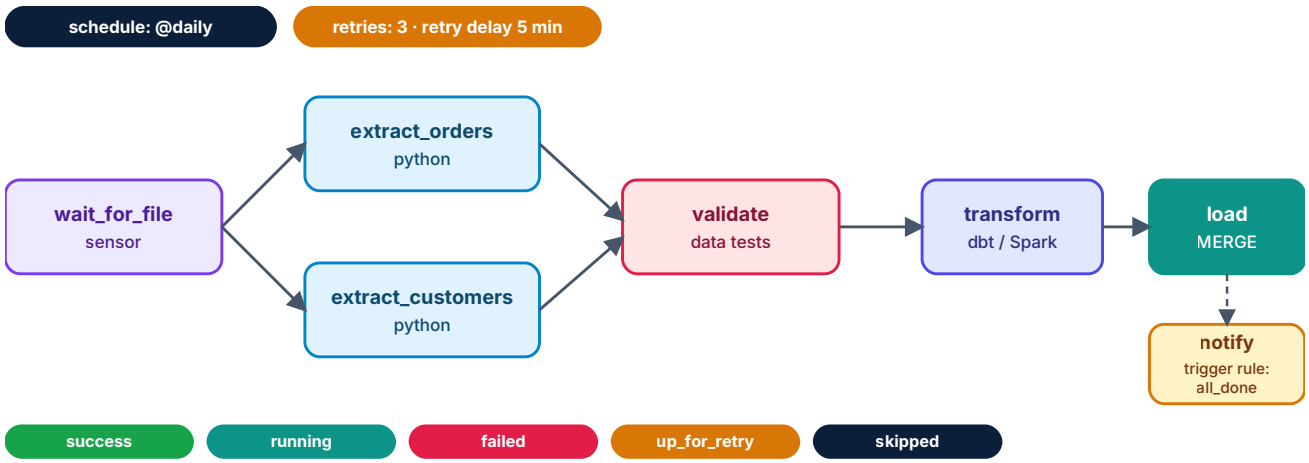


Figure 4.5 — An Airflow-style DAG: tasks, dependencies, states

Airflow vocabulary

- **DAG** — directed acyclic graph of tasks (no cycles).
- **Operator** / **task** — a unit of work; **sensor** waits for a condition.
- **Scheduler**, **executor**, **workers**, metadata DB.
- **XCom** — tiny messages between tasks (not big data!).
- **Catchup / backfill** — run past schedule intervals.

Good practices

- Make tasks **idempotent** and parameterised by logical date.
- Orchestrate, do not compute: push heavy work to Spark / the warehouse.
- Set retries, timeouts, SLAs and failure alerts.
- Keep DAG files light; no heavy code at import time.
- Orchestration (central scheduler) vs choreography (event-driven services).

4.6 Kafka & Message Brokers

★★★

IN ONE LINE

A durable, replayable event log that decouples producers from consumers.

Apache Kafka is a **distributed, durable, append-only commit log** used as the backbone for event streaming. Producers write events to **topics**; consumers read them at their own pace, and events are retained (days to forever) so they can be replayed.

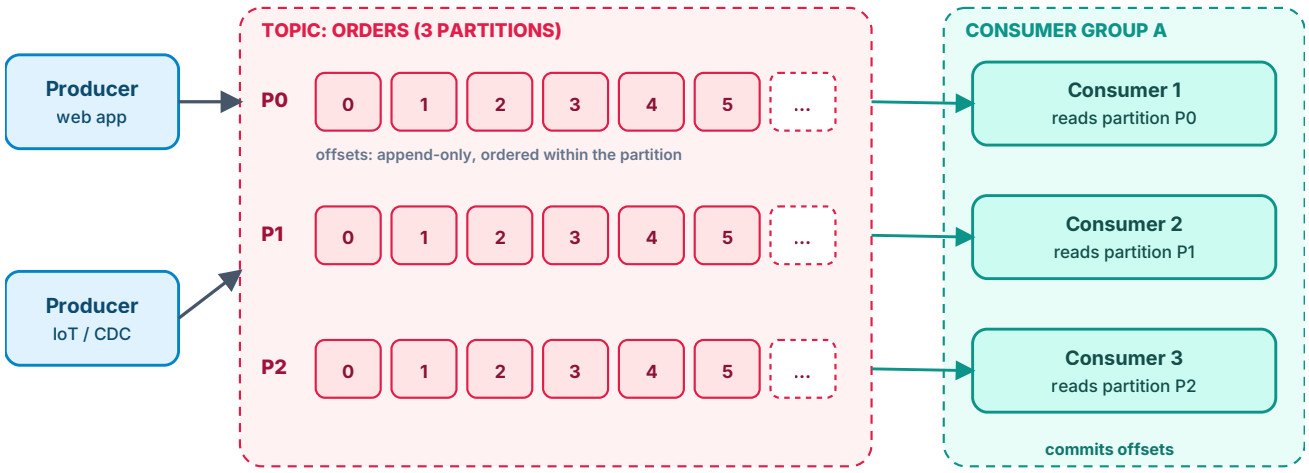


Figure 4.6 — Topics, partitions, offsets and consumer groups

Concept	Meaning
Topic / partition	A topic is split into partitions for parallelism; **ordering is guaranteed only within a partition**.
Key	Events with the same key go to the same partition (e.g. keep all events of one customer in order).
Offset	Position of a record in a partition; consumers track and commit it, enabling replay and restart.
Consumer group	Consumers share partitions (max 1 consumer per partition in a group). Different groups read independently.
Replication / ISR	Each partition has a leader and followers; acks=all with in-sync replicas gives durability.
Retention / compaction	Delete by age/size, or *compact* to keep the latest value per key (changelog topics).

INTERVIEW ANGLE

Kafka vs a queue (RabbitMQ/SQS): Kafka is a **log** — messages are not removed when read, multiple groups can consume, and you can replay. This ties directly to Kappa architecture and CDC.

4.7 CAP Theorem, Replication & Sharding

★★☆

IN ONE LINE

Fundamental trade-offs when data is spread across machines.

The **CAP theorem**: during a network **P**artition, a distributed data store must choose between **C**onsistency (every read sees the latest write) and **A**vailability (every request gets a response). Since partitions are unavoidable in distributed systems, the real choice is **CP vs AP**.

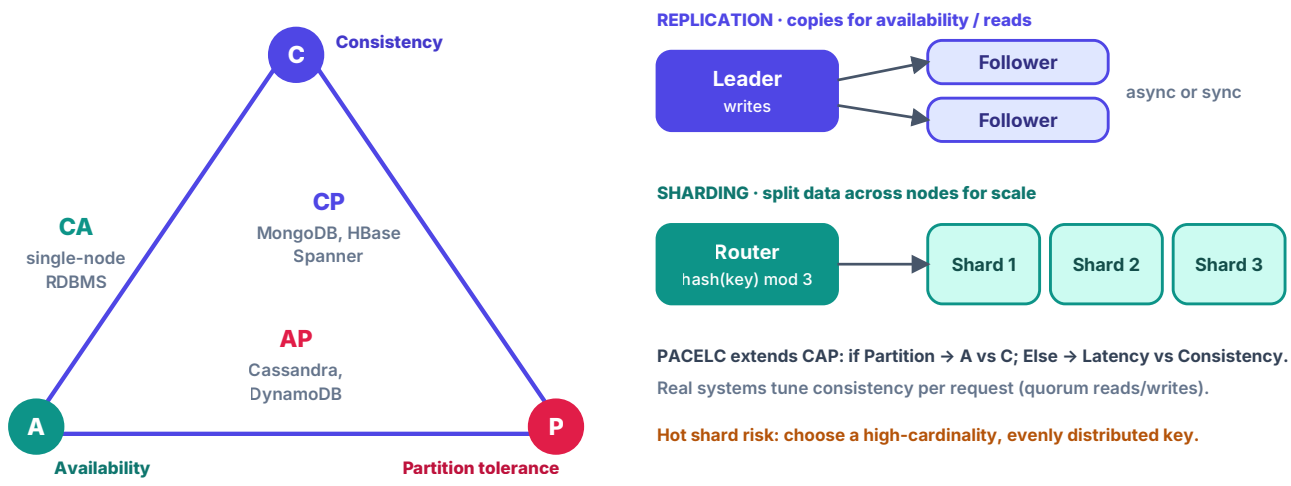


Figure 4.7 — CAP, replication and sharding

Term	Meaning	Why interviewers ask
Replication	Keep copies of data on multiple nodes (leader–follower or multi-leader)	Availability, read scaling, durability, lag / consistency trade-offs
Sharding / partitioning	Split data horizontally across nodes by key	Write scaling; hot keys; re-sharding cost
Eventual consistency	Replicas converge if writes stop	BASE systems, DynamoDB / Cassandra reads
Quorum	R + W > N guarantees overlap of read and write sets	Tunable consistency

REMEMBER

Remember the mix-up from 1.2: **C in CAP** is about replicas agreeing; **C in ACID** is about data satisfying integrity rules.

4.8 Data Vault, Surrogate Keys & Modeling Choices

★★☆

IN ONE LINE

An audit-friendly integration model, plus the key-design ideas behind every warehouse.

Data Vault separates **business keys** (Hubs), **relationships** (Links) and **descriptive history** (Satellites). It is built for auditability, parallel loading and absorbing source changes without redesign. It usually feeds dimensional marts on top.

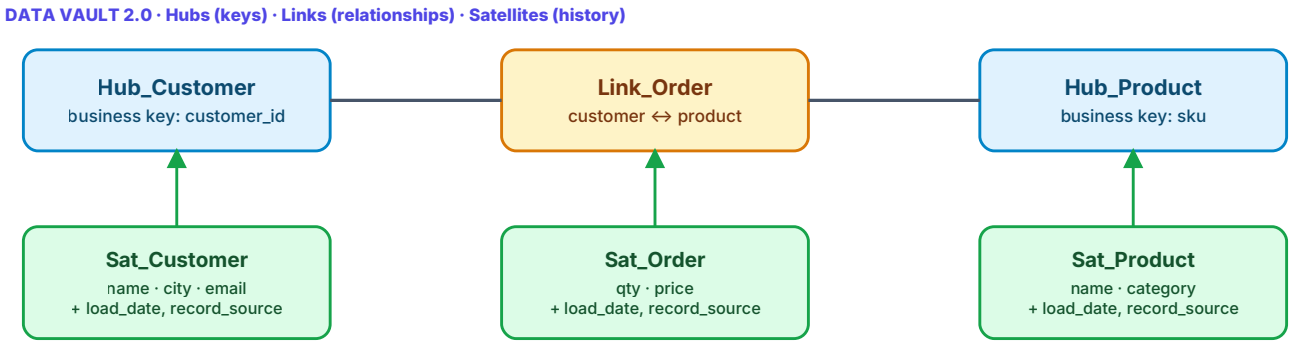


Figure 4.8 — Hubs, Links and Satellites

Surrogate vs natural keys

- **Natural / business key**: meaningful, from the source (email, SKU) — can change or collide across systems.
- **Surrogate key**: system-generated integer or hash with no meaning — stable, joins fast, insulates the warehouse from source changes.
- Hash keys (MD5 / SHA of business key) allow parallel, order-independent loads.

When to use what

- **Star schema**: BI-facing, simplicity and speed.
- **Data Vault**: large enterprises with many changing sources, strict audit needs.
- **One Big Table**: ML features and single-purpose dashboards.

4.9 Performance Tuning Cheat Sheet

★★★

IN ONE LINE

A symptom → cause → fix map for slow and expensive pipelines.

A compact troubleshooting map. Interviewers love "this job is slow — what do you check?". Start from the **Spark UI / query profile** and work through the symptoms.

Symptom	Likely cause	First fixes
Reading far more data than needed	No pruning; `SELECT *`	Select only needed columns; filter on partition / clustered columns; use columnar formats
Thousands of tiny tasks / slow listing	Small-file problem	`OPTIMIZE`, auto-compaction, larger micro-batch interval, fewer partitions
One task runs far longer than the rest	Data skew	AQE skew join, salting, filter null keys
Huge shuffle read / write	Wide transformations on big data	Broadcast small tables, pre-aggregate, filter early, reduce columns
Out-of-memory / spill	Large partitions, big collect, wide rows	More partitions, bigger executors, avoid `collect()` / `toPandas()` on big data
Slow MERGE	Scans whole target	Add partition / cluster predicates to `ON`, dedupe source, compact target
Slow Python code	Row-wise UDFs	Built-in functions, pandas / Arrow UDFs, push logic to SQL
Costs keep rising	Over-provisioned or always-on clusters	Auto-scaling, spot / serverless, auto-terminate, `availableNow` triggers, cache only what is reused

INTERVIEW ANGLE

Structure your answer: **measure → locate the bottleneck stage → identify cause (IO, shuffle, skew, spill, small files) → apply the targeted fix → re-measure**. Never start by "add more nodes".

4.10 How to Answer a Pipeline Design Question

★★★

IN ONE LINE

A repeatable eight-step framework for system-design style interview questions.

System-design questions ("design a pipeline for X") reward a **repeatable structure**. Walk through these eight steps out loud, and tie each to the concepts in this handbook.

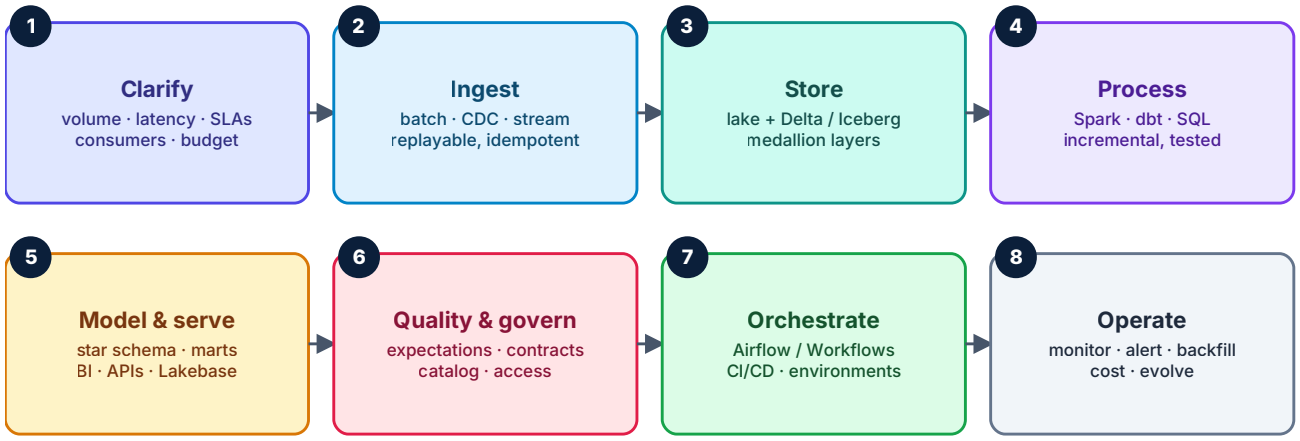


Figure 4.9 — An 8-step template for data-pipeline design answers

Step	Questions to ask / things to mention
1 Clarify	Data volume and growth? Freshness (latency) requirement? Consumers? Accuracy vs speed? Compliance (PII)? Budget?
2 Ingest	Full vs **incremental**; **CDC** for databases; Kafka for events; **idempotent** and replayable; handle late data (**watermarks**, lookback).
3 Store	Lakehouse with **Delta / Iceberg**, **Parquet**, **medallion** layers, sensible partitioning / clustering.
4 Process	Spark or dbt/SQL; **MERGE** for upserts; SCD2 where history matters; avoid shuffles and skew.
5 Model & serve	Star schema in Gold; marts per domain; BI via SQL warehouse; low-latency serving store (e.g. Lakebase / Postgres) if apps need it.
6 Quality & governance	Expectations / dbt tests, contracts, quarantine, lineage, catalog, row/column security.
7 Orchestrate & ship	DAG with retries, backfills; CI/CD with dev/stage/prod; infrastructure as code.
8 Operate	Freshness / volume alerts, SLAs, runbooks, cost control, schema evolution plan.

INTERVIEW ANGLE

Close with **trade-offs and alternatives** ("if latency were seconds I would add Kafka + streaming; if volume were small I would drop Spark for plain SQL"). Showing you can *change* the design when requirements change is what interviewers score highest.

4.11 Rapid-Fire Interview Q&A

★★★

IN ONE LINE

Twenty questions that cover most of the handbook, with answers short enough to say out loud.

Q What is the difference between a data lake, warehouse and lakehouse?

Warehouse: structured, schema-on-write, fast BI, higher cost. Lake: cheap storage of any data, schema-on-read, no ACID. Lakehouse: lake storage + open table format (Delta/Iceberg) giving ACID, governance and BI performance on one copy of data.

Q How does a WAL guarantee durability?

Changes are appended to a sequential log and fsynced at COMMIT before the client is acknowledged. Data pages are flushed later; after a crash the DB replays the WAL (REDO) and rolls back uncommitted work.

Q How does Delta Lake provide ACID on S3?

Data files are immutable Parquet; the _delta_log records add/remove actions as numbered commits. A commit is the atomic creation of the next log file (optimistic concurrency). Readers only trust files in the committed snapshot.

Q Parquet vs Delta?

Parquet is a columnar file format. Delta is a table format: Parquet files + a transaction log giving ACID, time travel, schema enforcement and MERGE.

Q Star vs snowflake schema?

Star: denormalised dimensions, fewer joins, faster and simpler. Snowflake: normalised dimensions, less redundancy, more joins. Star is the default for analytics.

Q Explain SCD Type 2.

Keep history by expiring the old dimension row (valid_to, is_current=false) and inserting a new row with a new surrogate key. Facts join to the version that was current at event time.

Q ETL vs ELT?

ETL transforms before loading in a separate engine; ELT loads raw data first and transforms inside the warehouse/lakehouse, keeping raw data for reprocessing.

Q What is idempotency and how do you achieve it?

Re-running gives the same result. Use MERGE on stable keys, partition overwrite, dedupe, processed-file tracking and deterministic transforms.

Q Can you get exactly-once?

Effectively yes: at-least-once delivery plus idempotent or transactional sinks and checkpointed offsets (Kafka transactions, Spark checkpoints + Delta). External non-idempotent systems need your own idempotency keys.

Q Two meanings of watermark?

Batch: stored last-loaded timestamp/ID for incremental loads. Streaming: event-time threshold (max event time minus allowed lateness) that decides when windows close and late data is dropped.

Q How do you handle late-arriving data?

Streaming watermarks with allowed lateness; batch lookback window reprocessing the last N days with MERGE/ partition overwrite; placeholder members for late dimensions.

Q Why is CDC better than polling updated_at?

Log-based CDC captures deletes and every intermediate change, preserves ordering and adds almost no load on the source. Polling misses hard deletes and can lose rows at boundaries.

Q Lambda vs Kappa?

Lambda: separate batch and speed layers merged at serving (two codebases). Kappa: one streaming pipeline on a replayable log; reprocess by replay. Lakehouse tables often unify both.

Q What is the small-file problem?

Many tiny files cause slow listing, high metadata overhead and too many tasks. Fix with compaction (OPTIMIZE), better partitioning, larger batch intervals and auto-compaction.

Q Narrow vs wide transformations?

Narrow: no data movement (filter, map). Wide: shuffle across the network (groupBy, join, distinct) and define stage boundaries.

Q How do you fix data skew?

AQE skew-join handling, salting hot keys, broadcast the small side, filter null/default keys, better partition keys.

Q What does the medallion architecture buy you?

Progressive quality layers: Bronze raw and replayable, Silver cleaned and conformed, Gold business-ready. Easier reprocessing, clearer contracts, tiered access.

Q Data mesh vs data fabric?

Mesh is an organisational model (domain ownership, data products, federated governance). Fabric is a technology approach (active metadata + automation integrating distributed data). Complementary.

Q What is Lakebase?

Databricks' managed serverless Postgres OLTP database integrated with the lakehouse (Neon-based), synced with Delta tables so apps and analytics share governed data.

Q How would you test a data pipeline?

Unit tests on transforms, data tests (unique, not_null, relationships), CI in an isolated schema, regression / data-diff before promotion, freshness and volume monitoring in production.

4.12 Glossary: Every Term in One Line

★★☆

IN ONE LINE

A quick-revision index of all the concepts in this handbook.

Every term from your list, plus the extras, in one place. The last column points to the section with the full explanation and diagram.

Term	One-line definition	§
ACID	Atomicity, Consistency, Isolation, Durability: transaction guarantees.	1.2
Atomicity	A transaction fully commits or has no effect.	1.3
Back-date refresh	Re-processing past periods for late data, fixes or new logic.	3.4
Batch processing	Processing bounded data on a schedule.	3.8
Bronze / Silver / Gold	Raw, cleaned and business-ready lakehouse layers.	2.12
CDC	Capturing row-level inserts, updates, deletes from a source.	3.10
CI/CD	Automated build, test and deployment of code.	3.13
CRUD	Create, Read, Update, Delete.	1.1
Data contract	Producer–consumer agreement on schema, semantics and SLAs.	4.3
Data fabric	Metadata-driven automated integration layer over distributed data.	2.11
Data lake	Cheap object storage for raw data of any type; schema-on-read.	2.2
Data lakehouse	Lake storage + table format giving warehouse reliability and speed.	2.3
Data mart	Subject-specific slice of analytical data.	2.9
Data mesh	Decentralised, domain-owned data products on a self-serve platform.	2.10
Data modeling	Designing data structures from concept to physical tables.	1.9
Data Ops	DevOps-style automation, testing and monitoring for data.	3.13
Data warehouse	Modelled, integrated, historical store for analytics.	2.1
dbt	SQL transformation framework with tests, docs and lineage.	3.12
Delta Lake	Open table format: Parquet + transaction log.	2.5
Delta Live Tables	Declarative pipelines; now Lakeflow Spark Declarative Pipelines.	3.11
Dimensional modeling	Fact and dimension tables around a business-process grain.	1.10
ELT	Extract, Load, then transform inside the platform.	3.1
ETL	Extract, Transform, then load.	3.1
Exactly-once	Each record's effect applied once despite retries.	3.6
Idempotency	Repeated runs give the same result.	3.5
Incremental loading	Loading only new or changed data.	3.2
Kafka	Distributed, replayable event log.	4.6
Kappa architecture	Single streaming pipeline over a replayable log.	3.9

Term	One-line definition	§
Lakebase	Databricks serverless Postgres OLTP integrated with the lakehouse.	2.8
Lambda architecture	Parallel batch and speed layers merged at serving.	3.9
Medallion architecture	Bronze → Silver → Gold refinement layers.	2.12
Normalization	Removing redundancy by splitting tables (1NF–3NF).	1.8
OLAP	Analytical, read-heavy, large-scan workloads.	1.7
OLTP	Transactional, many small concurrent operations.	1.7
Open table format	Metadata layer giving ACID tables on files (Delta, Iceberg, Hudi).	2.5
Parquet	Columnar, compressed file format for analytics.	2.4
Partitioning	Splitting data into folders for pruning.	4.1
Pipeline	Automated flow moving and transforming data.	3.1
Snowflake schema	Star schema with normalised dimensions.	1.10
Spark	Distributed in-memory data processing engine.	2.6
SCD	Slowly Changing Dimensions: managing attribute history.	1.11
Star schema	Central fact table with denormalised dimensions.	1.10
Streaming processing	Processing unbounded events continuously.	3.8
Transaction log	Ordered record of table changes (e.g. _delta_log).	2.5
Trino	Distributed MPP SQL engine for federated queries.	2.7
Upsert	Update if exists, otherwise insert (MERGE).	3.7
WAL	Write-Ahead Log: log changes before data files.	1.4
WAL buffer	In-memory staging area for WAL records before flush.	1.4
Watermark	Last loaded value (batch) or event-time progress marker (streaming).	3.3